

Corrispondenza tra Ordinamento dei Tensori e Layout di Memoria delle Matrici

Questo documento descrive formalmente la corrispondenza tra le nozioni tensoriali di *ordinamento naturale* e *ordinamento inverso* (definiti nel capitolo 1 della tesi) e i layout di memoria standard per matrici bidimensionali, noti come *column-major* e *row-major order*.

1 Formalizzazione per Matrici (Tensori 2-dimensionali)

Sia data una matrice $\mathbf{A} \in \mathbb{R}^{n_1 \times n_2}$, indicizzata dalla coppia di indici (i_1, i_2) , dove $i_1 \in [n_1]$ indica la riga e $i_2 \in [n_2]$ indica la colonna.

1.1 Ordinamento Naturale e Column-Major

L'ordinamento naturale $\mathbb{L}: [n_1] \times \cdots \times [n_d] \rightarrow [\prod_{j=1}^d n_j]$ definisce la vettorizzazione scorrendo per prima la prima dimensione. Per $d = 2$:

- I passi (o strides) sono definiti come $s_1 = 1$ e $s_2 = n_1$.
- La mappa di linearizzazione è:

$$\mathbb{L}(i_1, i_2) = i_1 + n_1(i_2 - 1).$$

Poiché il passo associato all'indice di riga i_1 è $s_1 = 1$, tale indice varia più velocemente. In memoria, gli elementi adiacenti appartengono alla stessa colonna:

$$\mathbb{L}(1, 1) = 1, \quad \mathbb{L}(2, 1) = 2, \quad \dots, \quad \mathbb{L}(n_1, 1) = n_1, \quad \mathbb{L}(1, 2) = n_1 + 1, \quad \dots$$

Questo corrisponde esattamente al layout **column-major order** (tipico di Fortran, MATLAB, R, Julia).

- **Vettorizzazione:** L'operatore $\vec{(\mathbf{A})}$ basato su $\mathbb{L}$ effettua un incolonnamento per colonne (ad esempio, $\mathbf{A}(:)$ in MATLAB o $\mathbf{A.flatten('F')}$ in Python).

1.2 Ordinamento Inverso e Row-Major

L'ordinamento inverso (reverse ordering) $\mathbb{L}^*: [n_1] \times \cdots \times [n_d] \rightarrow [\prod_{j=1}^d n_j]$ inverte l'ordine dei passi. Per $d = 2$:

- I passi sono definiti come $s_2^* = 1$ e $s_1^* = n_2$.
- La mappa di linearizzazione è:

$$\mathbb{L}^*(i_1, i_2) = i_2 + n_2(i_1 - 1).$$

Poiché il passo associato all'indice di colonna i_2 è $s_2^* = 1$, tale indice varia più velocemente. In memoria, gli elementi adiacenti appartengono alla stessa riga:

$$\mathbb{L}^*(1, 1) = 1, \quad \mathbb{L}^*(1, 2) = 2, \quad \dots, \quad \mathbb{L}^*(1, n_2) = n_2, \quad \mathbb{L}^*(2, 1) = n_2 + 1, \dots$$

Questo corrisponde esattamente al layout **row-major order** (tipico di C, C++, Python/NumPy/PyTorch per impostazione predefinita).

- **Vettorizzazione:** L'operazione di vettorizzazione associata a $\mathbb{L}^*$ corrisponde a scorrere la matrice riga per riga (ad esempio, `A.flatten('C')` in Python).

2 Estensione a Tensori multidimensionali ($d > 2$)

Sia $\mathcal{X} \in \mathbb{R}^{n_1 \times \cdots \times n_d}$ un tensore d-dimensionale indicizzato da $(i_1, i_2, \dots, i_d) \in [n_1] \times \cdots \times [n_d]$.

Ordinamento Naturale ($\mathbb{L}$): I passi sono $s_1 = 1$, $s_{j+1} = s_j n_j = n_1 \cdots n_j$. L'indice più a sinistra (i_1) varia più velocemente in memoria, mentre l'indice più a destra (i_d) varia più lentamente. Corrisponde a un layout multidimensionale di tipo **column-major** (Fortran).

Ordinamento Inverso ($\mathbb{L}^*$): I passi sono $s_d^* = 1$, $s_{j-1}^* = s_j^* n_j = n_j \cdots n_d$. L'indice più a destra (i_d) varia più velocemente in memoria, mentre l'indice più a sinistra (i_1) varia più lentamente. Corrisponde a un layout multidimensionale di tipo **row-major** (C, Python).

3 Tabella di Confronto

Ordinamento	Strides ($d = 2$)	Layout Matrice	Indice Veloce	Standard Informatici
Naturale ($\mathbb{L}$)	$s_1 = 1, s_2 = n_1$	Column-major	i_1 (riga)	MATLAB, Julia, R, Fortran
Inverso ($\mathbb{L}^*$)	$s_2^* = 1, s_1^* = n_2$	Row-major	i_2 (colonna)	C, C++, Python (NumPy)

Tabella 1: Confronto tra ordinamenti tensoriali e layout di memoria matriciali.